Smalltalk/CREXX

A Smalltalk-80 interpreter written in CREXX

Language Reference and User's Guide

Release 1 • Build 1
for CREXX 1.0.0 beta 3 and later

Publication STC-0001-0
File No. SMALLTALK-01

First Edition (October 2026)

This edition applies to Smalltalk/CREXX Release 1 (build 1, 10 October 2026), an interpreter for the Smalltalk-80 language written in CREXX Level B, and to all subsequent builds until otherwise indicated in new editions.

Smalltalk/CREXX is a hobbyist project. It is not a product of, and this manual is not a publication of, the authors or owners of any Smalltalk implementation. Smalltalk-80 and GNU Smalltalk are names of a language and a product, used here only to identify the language that this software implements and the implementation whose source syntax it follows.

CREXX is an open-source REXX implementation and is maintained by its own community. Smalltalk/CREXX needs it to run.

The project repository holds the interpreter source, the examples and the regression tests from which every statement in this manual can be checked. Comments on this publication may be addressed to the project.

Contents

Preface ... 1

Chapter 1. Introduction ... 2
- How Smalltalk/CREXX is built ... 2
- Status ... 2

Chapter 2. Installing and Running ... 4
- What you need ... 4
- Installation steps ... 4
- Running a program ... 4
- The interactive prompt ... 4
- Program arguments ... 5
- Starting the interpreter ... 5
- The regression tests ... 5

Chapter 3. The Smalltalk Language ... 6
- Comments and literals ... 6
- Variables ... 6
- Messages ... 6
- Cascades ... 7
- Assignment and statements ... 7
- Blocks ... 7
- Non-local return ... 7
- Control flow ... 8
- Top-level programs ... 8

Chapter 4. Classes and Objects ... 9
- Defining a class ... 9
- Reopening and extending classes ... 9
- Creating a class with messages ... 10
- Creating instances ... 10
- Inheritance and super ... 10
- Printing objects ... 11
- Metaclasses ... 11
- Class-side instance variables ... 12

Chapter 5. Numbers ... 13
- The number classes ... 13
- Arithmetic ... 13
- Large integers ... 13
- Floating-point numbers ... 14
- Reference ... 14
- More number methods ... 14
- Formatting numbers ... 15
- Complex numbers ... 15
- Infinity and NaN ... 15

Chapter 6. Collections ... 17
- The collection classes ... 17
- Protocol common to all collections ... 17
- Arrays and OrderedCollections ... 18
- Dictionaries ... 18
- Sets and Bags ... 19
- Intervals ... 19
- More collection methods ... 19
- Stack, Queue and Matrix ... 20
- LinkedList ... 20
- Printing order ... 21

Chapter 7. Strings, Symbols and Characters ... 22
- Strings ... 22
- Symbols ... 22
- Characters ... 23
- More string methods ... 23
- Wildcard matching ... 23
- JSON ... 24
- Regular expressions ... 24

Chapter 8. Streams and the Transcript ... 25
- The Transcript ... 25

Read streams.....25
Write streams.....25
Standard input.....25
Files.....26
Date and Time.....26
More stream methods.....27
Stopwatch.....27
Parsing and formatting dates.....27
ReadWriteStream.....27
Managing files and directories.....28
Running programs.....28
CSV.....28

Chapter 9. Networking.....30
Sockets.....30
URLs.....30
HTTP.....30

Chapter 10. Testing with SUnit.....32
Writing tests.....32
Running tests.....32

Chapter 11. Processes.....33
Starting processes.....33
Semaphores, queues and delays.....33

Chapter 12. Exceptions.....34
Handling.....34
Messages understood by an exception.....34
Signaling.....34
Cleaning up.....35
Your own exceptions.....35

Chapter 13. Reflection and the System.....36
Asking about objects.....36
Asking about classes.....36
The Smalltalk dictionary.....36
Contexts and compiling at run time.....37

Chapter 14. Errors and Traces.....38
Unhandled errors.....38
Common messages.....38
Parse errors.....38

Chapter 15. How Smalltalk/CREXX Works.....39
Objects.....39
Classes and methods.....39
The resolver.....39
The evaluator.....39
Primitives.....39
Adding a primitive.....39
The build.....40

Chapter 16. Restrictions and Known Problems.....41

Appendix A. Command Summary.....42

Appendix B. Contents of the Repository.....43

Appendix C. Class Hierarchy.....44

Appendix D. Sample Programs.....45

Preface

This manual introduces Smalltalk/CREXX, an interpreter for the Smalltalk-80 language written in CREXX. It tells you how to install and run the interpreter and describes the language it implements: the syntax of expressions and classes, the object model, the numeric tower, the collection classes, streams, exceptions and reflection.

It is written for the programmer who knows Smalltalk or another object-oriented language, and for the newcomer who wants to see what a pure object language looks like. Where Smalltalk/CREXX behaves as the classic systems do, this manual says little; where it differs, it says how.

How this manual is organized

- **Chapter 1, Introduction**, says what Smalltalk/CREXX is, how it is built and what it supports.
- **Chapter 2, Installing and Running**, covers the launcher, how programs are read and the regression tests.
- **Chapter 3, The Smalltalk Language**, describes literals, variables, the three kinds of message, cascades, blocks and control flow.
- **Chapter 4, Classes and Objects**, describes how classes are defined, class-side variables and methods, inheritance, super and metaclasses.
- **Chapter 5, Numbers**, covers integers of any size, fractions and floating-point numbers.
- **Chapter 6, Collections**, is the reference for Array, OrderedCollection, SortedCollection, Interval, Set, Bag and Dictionary.
- **Chapter 7, Strings, Symbols and Characters**, covers the text classes.
- **Chapter 8, Streams and the Transcript**, describes ReadStream, WriteStream and the Transcript.
- **Chapter 9, Networking**, covers sockets, URLs and the HTTP client.
- **Chapter 10, Testing with SUnit**, describes TestCase, TestSuite and TestResult.
- **Chapter 11, Processes**, describes Process, Processor, Semaphore, SharedQueue and Delay.
- **Chapter 12, Exceptions**, describes on:do:, ensure:, signal, retry, resume and the exception classes.
- **Chapter 13, Reflection and the System**, covers the Smalltalk dictionary, perform: and the class protocol.
- **Chapter 14, Errors and Traces**, shows what the interpreter prints for an unhandled error.
- **Chapter 15, How Smalltalk/CREXX Works**, describes the object table, the resolver, the evaluator and the primitives for those who wish to extend the interpreter.
- **Chapter 16, Restrictions and Known Problems**, lists what does not work yet.
- **Appendix A** summarizes the command line, **Appendix B** lists the files of the repository, **Appendix C** shows the class hierarchy, **Appendix D** lists the sample programs.

Conventions

Program text is shown in monospaced type. In syntax descriptions, words in italics stand for values you supply and items in square brackets [ ] are optional. Smalltalk names are case sensitive: Total and total are different variables. By convention class names begin with a capital letter and variable and message names with a lower-case letter.

Examples of programs are shown in framed figures. In a figure that shows a run, the output of the interpreter follows the program text.

Chapter 1. Introduction

Smalltalk/CREXX is a Smalltalk-80 interpreter written in CREXX, the open-source REXX implementation. It runs programs written in the bracket syntax of GNU Smalltalk: classes are declared with Object subclass: Foo [...], methods are written inside the class body, and statements at the top level are run one after the other. The interpreter is a single CREXX source file, smalltalk.crexx, written in CREXX Level B, the statically typed level of the language, and a class library written in Smalltalk itself.

Smalltalk/CREXX gives its users:

- **A pure object model.** Everything is an object: integers, characters, classes and blocks. All computation is by sending messages, and the only control structures are messages to booleans, blocks and numbers.
- **Classes and metaclasses.** Every class is an instance of its metaclass; class-side methods, class variables and class-side instance variables are all supported.
- **Blocks.** Blocks are closures. They may be stored, passed, and returned from methods, and a ^ inside a block returns from the method that defined it, even through several levels of calls.
- **Numbers without limits.** Integers grow to any size, division of integers gives exact fractions, and floating-point numbers are available for the rest.
- **A class library.** Collections, strings, symbols, streams and exceptions are provided by library classes written in Smalltalk, loaded when the interpreter starts.
- **Exceptions.** Handlers with return, retry, resume, pass and outer; ensure: and ifCurtailed: blocks that run as the stack unwinds; user-defined exception classes.

How Smalltalk/CREXX is built

A source file is read once and parsed statement by statement into a tree held in parallel arrays. A resolver then walks the tree: it turns variable names into frame slots, instance-variable positions or global cells, and compiles the standard control-flow messages (ifTrue:, whileTrue:, to:do:, and:, or:, timesRepeat:, ifNil: and their relatives) into direct nodes when their arguments are literal blocks. The evaluator walks the resolved tree and sends messages through a method cache.

Objects are entries in a table of parallel arrays. SmallIntegers are not objects in the table: they are encoded in the reference itself. Common operations on numbers, strings, arrays and dictionaries are implemented as primitives in CREXX; the class library calls them through the pragma <primitive: name> and falls back to Smalltalk code when a primitive fails. Chapter 15 describes the design.

The interpreter is compiled by the CREXX compiler to a bytecode file, which the CREXX virtual machine runs. The first run of the launcher performs this compilation; later runs start in about a second, of which most is the loading of the class library.

Status

The figure below shows the major areas of the language and their status in this release.

Area	Content	Status
Syntax	Expressions, cascades, blocks, literals, brace arrays, class-body syntax, Eval, message-style class creation	complete
Object model	Classes, metaclasses, instance and class variables, class-side instance variables, super, extend	complete
Control flow	Inlined conditionals and loops, non-local return	complete
Numbers	SmallInteger, large integers, Fraction, Float, mixed arithmetic	complete
Collections	Array, OrderedCollection, SortedCollection, Interval, Set, Bag, Dictionary, Association	complete
Text	String, Symbol, Character	complete
Streams	ReadStream, WriteStream, ReadWriteStream, FileStream, Transcript	complete
Date and Time	Date, Time, DateTime, calendar arithmetic, Time millisecondClock	complete

Area	Content	Status
Exceptions	on:do:, ensure:, ifCurtailed:, signal, retry, return:, resume:, pass, outer, exception sets	complete
Reflection	class, respondsTo:, perform:, selectors, instance variable names	most
Processes	Process, Semaphore, fork	not planned
Garbage collection	Objects are never freed	not planned
Bang (!) file format	The Smalltalk-80 chunk format	not planned

Figure 1. Smalltalk/CREXX language areas

Chapter 2. Installing and Running

What you need

Smalltalk/CREXX needs only the CREXX toolchain: the crexx driver together with the compiler (rxc), assembler (rxas) and virtual machine (rxvm) that it calls. CREXX 1.0.0 beta 3 was used to build and test the interpreter. The driver can be installed from a CREXX release package, or built from source with CMake.

The launcher script smalltalk is a POSIX shell script. The build script build.py needs Python 3, but only if you change the sources.

Installation steps

1. Install CREXX and make sure the crexx command is on your PATH. If it is not, set the environment variable CREXX to the full path of the driver.
2. Get the Smalltalk/CREXX repository (git clone https://github.com/mikebeer/smalltalk).
3. Run any program with the launcher: ./smalltalk examples/hello.st. The first run compiles the interpreter to bytecode and takes about half a minute. The compiled file, smalltalk.rxbin, is kept next to smalltalk.crexx.
4. Run the regression tests with ./tests/run_tests.sh. All of them should report ok.

Running a program

The launcher takes the names of source files. They are read and run in order, in one image: classes defined in the first file are visible in the second. With no file name the program is read from standard input. Output goes to standard output.

```
Transcript showCr: 'Hello, World!'.
(3 + 4) printNl.
#(1 2 3) printNl.
(1/3) + (1/6)

Hello, World!
7
(1 2 3 )
```

Figure 2. A first program, and its output

A top-level statement ends with a period. The value of a statement is discarded: unlike an interactive workspace, the interpreter does not print it. Use printNl or displayNl to show a value.

The interactive prompt

With the option -i the interpreter reads the named files, if any, and then shows a prompt st> on which it reads statements from standard input. The value of each expression is printed after it is evaluated; values that are nil or the Transcript are not shown. A statement that has an open bracket, parenthesis or quote continues on the next line, shown with the prompt ... . Type exit, quit or bye, or end the input, to leave. Temporary variables declared with | x | and classes defined at the prompt stay until the end of the session.

```
$ ./smalltalk -i
CREXX Smalltalk - type exit to leave
st> 3 + 4
7
st> (1 to: 3) collect: [:i | i * i]
(1 4 9 )
st> exit
```

Figure 3. An interactive session

Program arguments

Words after -- on the command line are not file names: they are handed to the program and answered by Smalltalk arguments as an Array of Strings. For example ./smalltalk report.st -- 2026 north runs report.st with the arguments ('2026' 'north').

Starting the interpreter

When it starts, the interpreter creates the kernel classes, then reads the library files lib/kernel.st, lib/numbers.st, lib/collections.st, lib/streams.st, lib/exceptions.st, lib/extras.st, lib/structures.st, lib/regex.st, lib/system.st, lib/net.st, lib/sunit.st and lib/process.st, and only then reads your program. The library directory is found by the launcher; with the driver you pass it as -L directory.

The regression tests

The directory tests holds a number of programs with the output they must produce. The script tests/run_tests.sh runs every one and compares the output, error trace included. The option --update rewrites the expected files after a deliberate change.

Chapter 3. The Smalltalk Language

Comments and literals

A comment is text enclosed in double quotes. Literals are the following.

Kind	Examples	Class
Integers	42 -7 16r1F 2r1010 1e3	SmallInteger or LargePositiveInteger
Large integers	12345678901234567890123456789O	LargePositiveInteger, LargeNegativeInteger
Floats	3.14 -0.5 2.5e3 1.0e-3	FloatD
Characters	$a $ $'	Character
Strings	'hello' 'it''s'	String
Symbols	#foo #at:put: #+ #'hello world'	Symbol
Literal arrays	#(1 2 #three $4 (5 6) foo true)	Array
Brace arrays	{1+1. 'two'. #three}	Array (evaluated)
Constants	nil true false	UndefinedObject, True, False

Figure 4. Literals

Inside a literal array, a bare word is a symbol, nil, true and false are the constants, and a parenthesized group is a nested array. A brace array is different: each element is an expression, evaluated when the brace array is evaluated.

Variables

A variable name is a letter followed by letters, digits and underscores. There are four kinds of variable.

- **Temporaries.** Declared between vertical bars, | a b |, at the head of a method, of a block, or at the top level of a program. The variables of a block are local to the block.
- **Instance variables.** Declared in a class body; each instance has its own copy.
- **Class variables.** Declared in a class body as Name := expression. They are shared by the class, its subclasses and its instances.
- **Globals.** Classes, and the variables created with Smalltalk at:put:. An assignment to a name that is not declared anywhere creates a global.

The pseudo-variables self, super, nil, true and false cannot be assigned to. self is the receiver of the current message; super is self, with method lookup starting in the superclass of the class that holds the method.

Messages

All computation is by sending messages. There are three forms, and they bind in this order: unary messages first, then binary, then keyword.

Kind	Form	Example
Unary	receiver selector	5 factorial
Binary	receiver op argument	3 + 4 a -> b x >= 0
Keyword	receiver key1: a key2: b	dict at: 1 put: 'one'

Figure 5. The three kinds of message

```
(2 + 3 * 4) printNl.
(2 + (3 * 4)) printNl.
(3 max: 4 + 5) printNl.
#(3 1 2) asSortedCollection asArray printNl.

20
14
9
(1 2 3 )
```

Figure 6. Precedence

Binary messages are evaluated from left to right with no precedence among operators, so 2 + 3 * 4 is 20. Parentheses override the order.

Cascades

A semicolon sends another message to the receiver of the previous message. The value of the cascade is the value of the last message.

```
| oc |
oc := OrderedCollection new.
oc add: 3; add: 1; add: 2; yourself.
oc printNl.

OrderedCollection (3 1 2 )
```

Figure 7. A cascade

Assignment and statements

An assignment is written name := expression. A statement ends with a period; the value of an assignment is the value assigned, so a := b := 0 works. A caret, ^ expression, returns from a method.

Blocks

A block is a sequence of statements in square brackets, with optional parameters: [:x :y | x + y]. A block is an object of class BlockClosure. It is evaluated with value, value:, value:value:, value:value:value:, value:value:value:value: or valueWithArguments:, according to the number of arguments; sending it the wrong number gives the error wrong argument count.

A block keeps the variables of the context in which it was created. The method in the next figure returns a block that adds a fixed amount.

```
Object subclass: Maker [
    adder: k [ ^[:x | x + k] ]
]
| add5 |
add5 := Maker new adder: 5.
(add5 value: 10) printNl.
(add5 value: 20) printNl.

15
25
```

Figure 8. A closure

Non-local return

A ^ inside a block returns from the method in which the block was written, not just from the block. The loop in the next figure ends as soon as the element is found.

```
Object subclass: Finder [
    find: x in: coll [
        coll do: [:e | e = x ifTrue: [^'found']].
        ^'missing'
    ]
]
(Finder new find: 3 in: #(1 2 3)) displayNl.
(Finder new find: 9 in: #(1 2 3)) displayNl.

found
missing
```

Figure 9. Return from inside a loop

If the method has already returned when such a block runs, the return cannot happen and the interpreter reports BlockContext>>cannotReturn: and abandons the statement.

Control flow

Smalltalk has no special syntax for conditions and loops. They are messages. When the arguments are literal blocks the resolver compiles them in line, so they are as fast as built-in statements; otherwise they are ordinary sends, defined in the library, so that the same messages work with blocks held in variables.

Message	Receiver	Meaning
ifTrue: ifFalse: ifTrue:ifFalse: ifFalse:ifTrue:	Boolean	Conditional; the value is the value of the chosen block, or nil
and: or: & | not xor:	Boolean	Logical operations; and: and or: do not evaluate the block if the result is known
whileTrue: whileFalse: whileTrue whileFalse	Block	Loop while the receiver block answers true (false)
repeat	Block	Loop forever; leave with ^ or an exception
to:do: to:by:do:	Number	Count from the receiver to the limit with a step
timesRepeat:	Integer	Evaluate the block that many times
ifNil: ifNotNil: ifNil:ifNotNil:	Any object	Test for nil; the ifNotNil: block may take the object as argument
do: collect: select: reject: detect: inject:into:	Collection	Iterate over the elements (Chapter 6)

Figure 10. Control-flow messages

```
1 to: 3 do: [:i | Transcript show: i printString; show: ' '].
Transcript cr.
3 to: 1 by: -1 do: [:i | Transcript show: i printString; show: ' '].
Transcript cr.
| n |
n := 1.
[n < 100] whileTrue: [n := n * 2].
n printNl.

1 2 3
3 2 1
128
```

Figure 11. Loops

Top-level programs

A program is a sequence of class definitions and statements. Variables declared at the top level with | a b | live until the end of the file. A statement that fails with an error prints a trace (Chapter 14) and the program continues with the next statement. The construct Eval [statements] groups statements; it is accepted for compatibility with GNU Smalltalk.

Chapter 4. Classes and Objects

Defining a class

A class is declared with the message subclass: in bracket form. The body of the brackets holds instance-variable declarations, class-variable declarations and methods.

If the name is already used by a class of the class library, for example Stack, Queue, Matrix or Complex, the declaration replaces that class: its methods and variables are discarded and the program's own definition applies. To add to a library class instead, use extend.

```
Object subclass: Account [
    | balance |
    Count := 0.
    Account class >> new [ Count := Count + 1. ^super new init ]
    Account class >> count [ ^Count ]
    init [ balance := 0 ]
    balance [ ^balance ]
    deposit: amount [
        amount <= 0 ifTrue: [^self error: 'Invalid amount'].
        balance := balance + amount
    ]
]
| a |
a := Account new.
a deposit: 50; deposit: 25.
a balance printNl.
Account new.
Account count printNl.
a deposit: -1.

75
2
Object: an Account error: Invalid amount
Account(Object)>>error:
Account>>deposit:
UndefinedObject>>executeStatements
```

Figure 12. A class with instance and class variables

Inside the body:

- | a b | declares instance variables.
- Name := expression. declares a class variable and gives it an initial value, which is computed once when the class is defined.
- selector [statements] defines an instance method. The pattern is a unary selector, a binary operator with one argument, or keywords with an argument each.
- Foo class >> selector [statements] defines a method on the class side. Inside the body of Foo, the class name may be written in this form.
- Foo class [| a b |] declares class-side instance variables, which each class (and each subclass) holds separately.
- <comment: 'text'> is accepted and ignored.

A method returns the value of the expression after ^. A method that does not execute a ^ returns self. Temporary variables of a method are declared at its start: | t1 t2 |.

Reopening and extending classes

Foo extend [...] adds methods and variables to an existing class, including the kernel classes, and Foo class extend [...] does the same for the class side. A single method may be added with Foo >> selector [...] or Foo class >> selector [...].

```
Integer extend [
    double [ ^self * 2 ]
    isPerfect [
        | sum |
        sum := 0.
        1 to: self - 1 do: [:i | self \\ i = 0 ifTrue: [sum := sum + i]].
        ^sum = self
    ]
]
5 double printNl.
((1 to: 500) select: [:n | n isPerfect]) printNl.

10
(6 28 496 )
```

Figure 13. Extending a kernel class

Methods added to a class are seen at once by every instance, including those created earlier. Instance variables added to a class are seen by new instances; instances that already exist are not changed.

Creating a class with messages

The classic form also works. The class is created by sending subclass:instanceVariableNames:classVariableNames:package: (or category:) to the superclass, and methods are added with extend.

```
Object subclass: #Pair
    instanceVariableNames: 'first second'
    classVariableNames: ''
    package: 'demo'.
Pair extend [
    first [ ^first ]
    first: x [ first := x ]
]
(Pair new first: 5; yourself) first printNl.

5
```

Figure 14. Message form

Creating instances

new and new: create instances. In this system new does not send initialize; the convention is to override the class-side new, as in the Account example above. new: n creates an indexable object with n slots, as for Array new: 5.

Inheritance and super

Messages are looked up in the class of the receiver, then in its superclass, and so on up to Object. If no method is found the message doesNotUnderstand: is sent to the receiver. A message to super looks up the method starting in the superclass of the class in which the method was written.

```
Object subclass: Animal [
    | name |
    Animal class >> named: aName [ ^self new setName: aName ]
    setName: aName [ name := aName ]
    speak [ ^self subclassResponsibility ]
    describe [ ^name, ' says ', self speak ]
]
Animal subclass: Dog [ speak [ ^'Woof' ] ]
Animal subclass: Cat [
    speak [ ^'Meow' ]
    describe [ ^super describe, '!' ]
]
(Dog named: 'Rex') describe displayNl.
(Cat named: 'Tom') describe displayNl.

Rex says Woof
Tom says Meow!
```

Figure 15. Inheritance

Printing objects

printNl prints printString and displayNl prints displayString. The printString of an object is built by sending it printOn: with a stream, so a class changes how it prints by overriding printOn:. displayString differs from printString for strings, symbols and characters (no quotes) and for collections (the elements are shown with displayString).

```
Object subclass: Point3 [
    | x y z |
    x: ax y: ay z: az [ x := ax. y := ay. z := az ]
    printOn: aStream [
        aStream nextPutAll: 'Point3('; print: x; nextPutAll: ', ';
            print: y; nextPutAll: ', '; print: z; nextPut: $)
    ]
]
(Point3 new x: 1 y: 2 z: 3) printNl.
(Array with: (Point3 new x: 0 y: 0 z: 0)) printNl.

Point3(1, 2, 3)
(Point3(0, 0, 0) )
```

Figure 16. Overriding printOn:

Metaclasses

A class is an object, and so has a class: its metaclass. The metaclass of Foo is written Foo class. The superclass of Object class is Class, so every class understands the messages of Class, Behavior and Object. Class-side methods are instance methods of the metaclass.

```
3 class printNl.
3 class class printNl.
3 class class class printNl.
3 class class class class printNl.
Integer superclass printNl.
Object superclass printNl.

SmallInteger
SmallInteger class
Metaclass
Metaclass class
Number
nil
```

Figure 17. Classes are objects

Class-side instance variables

Foo class [| count |] declares a variable that belongs to the class object itself. It is not shared with subclasses: each subclass has its own copy. This differs from a class variable, which is one cell shared by the class and all its subclasses.

Chapter 5. Numbers

The number classes

Number has the subclasses Integer, Fraction and Float. Integer has SmallInteger (up to 18 decimal digits) and LargePositiveInteger and LargeNegativeInteger. The transition between them is automatic and cannot be seen except by asking for the class. Float is represented by the class FloatD, a 64-bit floating-point number.

```
3 class printNl.
1000000000000 class printNl.
(2 raisedTo: 100) class printNl.
(2 raisedTo: 100) negated class printNl.
(1/2) class printNl.
3.5 class printNl.

SmallInteger
SmallInteger
LargePositiveInteger
LargeNegativeInteger
Fraction
FloatD
```

Figure 18. The number classes

Arithmetic

+, -, * and / are the four operations. Dividing two integers gives an integer if the division is exact and a Fraction otherwise; fractions are always reduced and a fraction with denominator 1 becomes an integer. // is integer division rounding toward minus infinity and \\ is the matching remainder; quo: and rem: round toward zero.

```
(7 / 2) printNl.
(8 / 2) printNl.
(7 // 2) printNl.
(-7 // 2) printNl.
(-7 \\ 2) printNl.
(-7 quo: 2) printNl.
(-7 rem: 2) printNl.
((1/3) + (1/6)) printNl.
((1/2) + 0.25) printNl.

7/2
4
3
-4
1
-3
-1
1/2
0.75
```

Figure 19. Division

When two numbers of different classes are combined, the one of lower generality is converted: SmallInteger and LargeInteger, then Fraction, then Float. Dividing by zero signals ZeroDivide.

Large integers

Large integers are held as decimal strings and arithmetic on them is done digit by digit in CREXX. They are exact but slower than SmallIntegers. Results that fit in a SmallInteger are always converted back.

```
30 factorial printNl.
(100 factorial / 98 factorial) printNl.
(2 raisedTo: 100) printNl.
(12345678901234567890 * 98765432109876543210) printNl.

265252859812191058636308480000000
9900
1267650600228229401496703205376
1219326311370217952237463801111263526900
```

Figure 20. Factorial

Floating-point numbers

Floats print with up to 15 significant digits and always show a decimal point. The functions sqrt, sin, cos, tan, arcTan, arcSin, arcCos, ln, log, log: and exp are available; raisedTo: with a fractional exponent gives a Float. Float pi and Float e give the constants.

Reference

Selector	Meaning
+ - * / // \\ quo: rem:	Arithmetic
< > <= >= = ~=	Comparison; = between a number and a non-number answers false
abs negated sign squared reciprocal	Unary arithmetic
even odd isZero isPrime isInteger isFloat isFraction isNumber	Tests
max: min: between:and:	Comparison helpers
truncated floor ceiling rounded roundTo: fractionPart integerPart	Conversion to integers
asFloat asInteger asString asCharacter printString printString: base radix: printPaddedWith:to:	Conversion and printing
sqrt sin cos tan arcTan arcSin arcCos ln log log: exp raisedTo: **	Mathematical functions
gcd: lcm: factorial	Integer functions
bitAnd: bitOr: bitXor: bitShift: << >> bitAt:	Bit operations on non-negative integers
to: to:by: to:do: to:by:do: timesRepeat:	Intervals and loops
@	Make a Point

Figure 21. Methods of Number

More number methods

Integers understand binomial:, fibonacci, floorLog:, isPerfect, isSquare, isPowerOfTwo, highBit, lowBit, bitInvert, noMask: and anyMask:. digits answers the decimal digits as an Array and sumOfDigits adds them. romanNumeral (also printRoman) answers a Roman numeral as a String. Any number understands degreesToRadians, radiansToDegrees and min:max:. to:collect: is a short form of collecting over an Interval.

```
10 fibonacci printNl.
(5 binomial: 2) printNl.
1994 romanNumeral printNl.
28 isPerfect printNl.
12345 digits printNl.
64 isPowerOfTwo printNl.
(15 min: 1 max: 10) printNl.

55
10
'MCMXCIV'
true
(1 2 3 4 5 )
true
10
```

Figure 22. More number methods

Formatting numbers

printShowingDecimalPlaces: n answers a number rounded to n decimal places as a String, and asStringWithCommas groups the digits of an Integer in threes. raisedTo: accepts a Fraction or a Float exponent.

```
(3.14159 printShowingDecimalPlaces: 2) printNl.
(2 printShowingDecimalPlaces: 2) printNl.
1234567 asStringWithCommas printNl.

'3.14'
'2.00'
'1,234,567'
```

Figure 23. Formatting numbers

Complex numbers

n i makes the imaginary number n times i, so 3 + 4 i is a Complex number. Complex numbers support + - * / and =, and answer real, imaginary, abs, conjugate and negated. They print as 3+4i and mix freely with integers, fractions and floats.

```
(3 + 4 i) printNl.
((3 + 4 i) * (1 - 2 i)) printNl.
(3 + 4 i) abs printNl.
(3 + 4 i) conjugate printNl.

3+4i
11-2i
5.0
3-4i
```

Figure 24. Complex arithmetic

Infinity and NaN

Float infinity, Float negativeInfinity and Float nan answer the special floating-point values; isInfinite and isNaN test for them. The square root of a negative number and the logarithm of a negative number answer NaN, and the logarithm of zero answers negative infinity. Float division by zero is still an error. truncated, floor, ceiling and rounded of an infinite or NaN value are errors.

```
-1 sqrt printNl.
Float infinity printNl.
Float negativeInfinity printNl.
0 ln printNl.
-1 sqrt isNaN printNl.

nan
inf
-inf
-inf
true
```

Figure 25. Special values

Chapter 6. Collections

The collection classes

All collections answer the messages of Collection (the figure below). The classes differ in how elements are stored and retrieved.

Class	Description
Array	Fixed size, indexed from 1. Literal: #(1 2 3). Created with Array new: n, with:with:, withAll:.
OrderedCollection	Grows at both ends: add:, addFirst:, removeFirst, removeLast, add:beforeIndex:, removeIndex:.
SortedCollection	An OrderedCollection that keeps its elements in the order of a sort block (default <=).
Interval	The numbers from a start to a stop with a step: 1 to: 10, 10 to: 1 by: -2. Elements are computed, not stored.
Set	Unordered collection without duplicates.
Bag	Collection that counts the number of occurrences of each element.
Dictionary	Maps keys to values. Association is a key-value pair.
String, Symbol	Collections of characters (Chapter 7).

Figure 26. The collection classes

Protocol common to all collections

Selector	Meaning
size isEmpty notEmpty includes: occurrencesOf:	Size and membership
do: do:separatedBy: reverseDo: doWithIndex: keysAndValuesDo: with:do:	Enumeration
collect: select: reject: detect: detect:ifNone: inject:into: fold:	Transforming and searching
anySatisfy: allSatisfy: count: detectMax: detectMin: max min sum	Testing and totals
asArray asOrderedCollection asBag asSet asSortedCollection asSortedCollection:	Conversion
add: addAll: remove: remove:ifAbsent: removeAll:	Changing (not for Array)
first last first: last: allButFirst allButLast at:ifAbsent: indexOf:	Position (sequenceable collections)
copyFrom:to: copyWith: copyWithout: , reverse copyReplaceAll:with:	Copying (sequenceable collections)
sort sort: sorted sorted: asSortedCollection:	Sorting

Figure 27. Methods of Collection

```
| a |
a := #(5 3 9 1).
(a collect: [:x | x * x]) printNl.
(a select: [:x | x > 2]) printNl.
(a inject: 0 into: [:s :x | s + x]) printNl.
(a detect: [:x | x > 100] ifNone: ['none']) printNl.
a asSortedCollection printNl.
(a asSortedCollection: [:x :y | x >= y]) asArray printNl.

(25 9 81 1 )
(5 3 9 )
18
'none'
SortedCollection (1 3 5 9 )
(9 5 3 1 )
```

Figure 28. Enumerating

collect: on an Array gives an Array, on an OrderedCollection an OrderedCollection, on a Set a Set, on a Dictionary a Bag, and on an Interval an Array. select: and reject: give a collection of the same kind as the receiver.

Arrays and OrderedCollections

Indexes start at 1. An index outside the collection is an error of class IndexOutOfRange (Chapter 12). OrderedCollection grows as needed and removes from either end in constant time; inserting or removing in the middle moves the elements after it.

```
| q |
q := OrderedCollection new.
q add: 1; add: 2; add: 3.
q addFirst: 0.
q removeFirst printNl.
q removeLast printNl.
q printNl.
q removeFirst; removeFirst.
q removeFirst.

0
3
OrderedCollection (1 2 )
Object: OrderedCollection () error: access beyond end of object
OrderedCollection>>removeFirst
UndefinedObject>>executeStatements
```

Figure 29. An OrderedCollection used as a queue

Dictionaries

Keys may be numbers, strings, symbols, characters, arrays of those, or any object (compared by identity). Dictionary keys, values and associations are enumerated in the order in which the keys were first added.

Selector	Meaning
at: at:put: at:ifAbsent: at:ifAbsentPut: at:ifPresent:	Access; at: of a missing key is an error
add: addAll:	Add an Association (or the contents of another dictionary)
removeKey: removeKey:ifAbsent:	Remove
includesKey: includes: keyAtValue: keyAtValue:ifAbsent:	Search
keys (a Set) values (an Array) associations	Parts
do: keysDo: keysAndValuesDo: associationsDo:	Enumeration; do: iterates over values
collect: (a Bag) select: reject: keysSelect:	Transformation; select: answers a Dictionary
size isEmpty copy	Size and copying

Figure 30. Methods of Dictionary

```
| d |
d := Dictionary new.
d at: 'one' put: 1; at: 'two' put: 2; at: 'three' put: 3.
d printNl.
(d at: 'two') printNl.
(d at: 'four' ifAbsent: [0]) printNl.
d removeKey: 'two'.
(d collect: [:v | v * 10]) printNl.
d keys asSortedCollection printNl.
d at: #missing.

a Dictionary ('one'->1 'three'->3 'two'->2 )
2
0
Bag (10 30 )
SortedCollection ('one' 'three' )
Object: #missing error: key not found
Dictionary>>at:
UndefinedObject>>executeStatements
```

Figure 31. A dictionary

Sets and Bags

```
| s b |
s := Set new.
s add: 1; add: 2; add: 1; add: 'a'; add: 'a'.
s size printNl.
s remove: 1.
(s includes: 1) printNl.
b := Bag new.
b add: 'x'; add: 'y'; add: 'x'.
(b occurrencesOf: 'x') printNl.
b size printNl.

3
false
2
3
```

Figure 32. Set and Bag

Intervals

1 to: 5 and 10 to: 1 by: -3 create Intervals. An Interval is a collection: it understands do:, collect:, select:, inject:into:, reverse, asArray and the rest. It prints like an Array.

More collection methods

Sequenceable collections add copyUpTo:, copyAfter:, copyUpToLast:, copyAfterLast:, lastIndexOf:, take:, drop:, fourth, middle, withIndexDo:, withIndexCollect:, pairsDo:, isSorted, atRandom, shuffled, zip:, cumulative, flatten, transpose, cartesianProduct:, combinations: and permutations. Every collection understands partition:, groupBy:, flatCollect:, detectSum:, detect:ifFound:, isSubsetOf:, product, range, variance, stdev, mode, median and asDictionary. An OrderedCollection answers removeFirst: and removeLast: for several elements at once. A Set offers union:, intersect:, difference: and symmetricDifference:, written also as |, & and -. A Dictionary offers keysSorted, merge:, invert, removeKeys: and associationAt:.

```
(#(1 2 3 4 5) take: 2) printNl.
(#(1 2 3 4 5) partition: [:x | x even]) printNl.
(#(1 2 3) zip: #(4 5 6)) printNl.
#(1 2 3 4) cumulative printNl.
(#(1 2 3 4) combinations: 2) printNl.
#(1 2 3) permutations size printNl.
#(3 1 2) median printNl.
#(1 #(2 #(3)) 4) flatten printNl.
| a b |
a := Set withAll: #(1 2 3). b := Set withAll: #(2 3 4).
(a & b) size printNl.
(a - b) printNl.

(1 2 )
((2 4 ) (1 3 5 ) )
((1 4 ) (2 5 ) (3 6 ) )
(1 3 6 10 )
((1 2 ) (1 3 ) (1 4 ) (2 3 ) (2 4 ) (3 4 ) )
6
2
(1 2 3 4 )
2
Set (1 )
```

Figure 33. Working with groups of elements

Stack, Queue and Matrix

Three small classes are written in Smalltalk. A Stack answers push:, pop, top, isEmpty, size and do: (from the top down). A Queue answers enqueue:, dequeue, front, isEmpty and size. Taking from an empty Stack or Queue is an error. A Matrix is made with rows:columns:, rows:columns:withAll:, identity: or fromRows:, and understands at:at:, at:at:put:, row:, column:, +, -, * (with a number or another Matrix), transposed, trace and determinant. It prints one row per line.

```
| s q m |
s := Stack new. s push: 1; push: 2; push: 3.
s pop printNl.
s printNl.
q := Queue new. q enqueue: 1; enqueue: 2.
q dequeue printNl.
m := Matrix fromRows: #(#(1 2) #(3 4)).
(m * m) printNl.
m determinant printNl.

3
Stack(1 2 )
1
[7 10]
[15 22]
-2
```

Figure 34. Stack, Queue and Matrix

LinkedList

A LinkedList is a doubly linked collection with fast addFirst:, addLast:, removeFirst and removeLast. It is a Collection, so do:, collect:, inject:into: and the rest work, and it also answers first, last, reverse, reverseDo: and remove:ifAbsent:.

```
| l |
l := LinkedList new. l add: 1; add: 2; addFirst: 0.
l printNl.
l removeFirst printNl.
l reverse printNl.

LinkedList(0 1 2 )
0
LinkedList(2 1 )
```

Figure 35. A LinkedList

Printing order

A Set, a Bag or a Dictionary has no order of its own, but it prints in sorted order whenever its elements (for a Dictionary, its keys) can be compared with <=, so the same program always prints the same text. do: and the other enumerators still visit the elements in the internal order. Elements that cannot be compared, such as mixed Strings and numbers, print in the internal order.

Chapter 7. Strings, Symbols and Characters

Strings

A String is an indexed collection of Characters. String literals are in single quotes, with an embedded quote written twice. Strings are mutable: at:put: changes a character in place. copy makes a separate string. The comparison messages <, <=, > and >= compare character codes; = compares the characters and is false between a String and a Symbol with the same letters.

Selector	Meaning
size isEmpty notEmpty first last at: at:put:	Basic access
, copyFrom:to: copyFrom: first: last: copyWith:	Joining and slicing
asUppercase asLowercase reverse reversed trimSeparators	Transforming
indexOf: occurrencesOf: includes: includesSubstring: startsWith: endsWith: indexOfSubCollection:	Searching
copyReplaceAll:with: replaceAll:with: copyReplacing:with: bindWith: %	Replacing
subStrings subStrings: lines tokenize:	Splitting; the result is an Array of Strings
asSymbol asString asNumber asInteger asArray asBag asSet asOrderedCollection	Conversion; asNumber and asInteger answer nil for text that is not a number
do: collect: select: detect: inject:into: reverseDo: keysAndValuesDo:	Enumeration; collect: must answer Characters
displayString printString displayNl printNl	Printing: displayString has no quotes
join:	The receiver is put between the elements of the argument

Figure 36. Methods of String

```
| s |
s := 'Hello, World'.
s size printNl.
s asUppercase printNl.
s reverse printNl.
(s copyFrom: 1 to: 5) printNl.
(s indexOf: $o) printNl.
(s subStrings: ', ') printNl.
(s copyReplaceAll: 'World' with: 'There') printNl.
('hello' select: [:c | c isVowel]) printNl.
'it''s' printNl.
'it''s' displayNl.

12
'HELLO, WORLD'
'dlroW ,olleH'
'Hello'
5
('Hello' 'World' )
'Hello, There'
'eo'
'it''s'
it's
```

Figure 37. Strings

Symbols

A Symbol is a String that is unique: two symbols with the same characters are the same object, so symbols compare with == and are cheap to use as keys. #foo, #at:put: and #+ are literals; asSymbol makes a symbol from a string. A symbol prints with a leading #, and with quotes when it has characters that a selector cannot

have. Symbols cannot be changed. A Symbol answers value: by sending itself, so #(1 2 3) collect: #squared works.

Characters

A Character is written $a. Character value: n gives the character with code n; Character nl, tab, space and cr give the usual ones. A Character answers value (the code), asUppercase, asLowercase, isLetter, isDigit, isVowel, isUppercase, isLowercase, isSeparator, digitValue and asString, and compares with <, >, <= and >=. Characters are unique objects.

```
$a printNl.
$a value printNl.
97 asCharacter printNl.
$a asUppercase printNl.
$5 digitValue printNl.
Character nl printNl.
Character space printNl.

$a
97
$a
$A
5
Character value: 10
$
```

Figure 38. Characters

More string methods

copyUpTo:, copyAfter:, copyUpToLast: and copyAfterLast: cut a string at a character. asUppercaseFirst (also capitalized) raises the first letter. padLeft:to: and padRight:to: pad with a given character, and padLeftTo: and padRightTo: pad with spaces. format: replaces {1}, {2} and so on with the elements of an Array, or {name} with the values of a Dictionary keyed by Symbols.

```
('a:b:c' copyUpTo: $:) printNl.
('a:b:c' copyAfterLast: $:) printNl.
'hello' capitalized printNl.
('7' padLeft: $0 to: 3) printNl.
('{1} is {2}' format: #('x' 42)) printNl.

'a'
'c'
'Hello'
'007'
'x is 42'
```

Figure 39. Cutting, padding and formatting

Wildcard matching

pattern match: text compares a string against a pattern in which * stands for any run of characters and # for any one character. The comparison ignores case.

```
('h*o' match: 'hello') printNl.
('h#llo' match: 'hello') printNl.
('h*x' match: 'hello') printNl.

true
true
false
```

Figure 40. Matching

JSON

asJson answers the JSON text for nil, Booleans, numbers, Strings, Symbols, collections and Dictionaries, with Dictionary keys in sorted order. fromJson reads JSON text and answers Dictionaries, Arrays, Strings, numbers, Booleans and nil. Malformed text is an error.

```
| d j |
d := Dictionary new.
d at: 'name' put: 'Ann'; at: 'tags' put: #('a' 'b'); at: 'ok' put: true.
d asJson displayNl.
j := '{"a": [1, 2.5, null], "b": {"c": -3}}' fromJson.
(j at: 'a') printNl.
((j at: 'b') at: 'c') printNl.

{"name":"Ann","ok":true,"tags":["a","b"]}
(1 2.5 nil )
-3
```

Figure 41. JSON

Regular expressions

A regular expression engine is written in Smalltalk. A pattern may use literal characters, . (any character but a newline), character classes such as [a-z0-9_] and [^x], the escapes \d \w \s and their capital forms for the complements, \b for a word boundary, the anchors ^ and $, groups ( ) and (?:), alternation |, and the repetitions * + ? {n} {n,} {n,m}, each of which becomes lazy when followed by ?. To use a metacharacter literally, put a backslash before it.

matchesRegex: tests the whole string. searchRegex: answers a RegexMatch for the first match, or nil; the match answers matched, start, stop and group: n. regexMatch: answers the first matching text, allRegexMatches: answers all of them as an Array, and occurrencesOfRegex: counts them. copyReplacingRegex:with: replaces every match, where \1 to \9 in the replacement stand for the groups, and splitOnRegex: cuts a string at each match. A pattern with a missing) or] is an error. The matcher is recursive, so very long texts matched by a repetition such as .* use a deep stack.

```
('abc123' matchesRegex: '[a-z]+\d+') printNl.
('foo 12 bar 345' allRegexMatches: '\d+') printNl.
(('2026-10-10' searchRegex: '(\d+)-(\d+)-(\d+)') group: 2) printNl.
('John Smith' copyReplacingRegex: '(\w+) (\w+)' with: '\2, \1') printNl.
('a1b22c' splitOnRegex: '\d+') printNl.
('<a><b>' regexMatch: '<.+?>') printNl.

true
('12' '345' )
'10'
'Smith, John'
('a' 'b' 'c' )
'<a>'
```

Figure 42. Regular expressions

Chapter 8. Streams and the Transcript

The Transcript

Transcript is the standard output. It understands show:, showCr:, display:, print:, displayNl:, nl, cr, tab, space, tab: and space:, and nextPutAll: and nextPut: like any stream. show: takes a String; display: and print: take any object. Output printed with printNl and displayNl goes to the same place, in order.

```
Transcript show: 'Total: '; show: 42 printString; cr.
Transcript display: 'text'; tab; print: 'text'; cr.
Transcript showCr: 'done'.

Total: 42
text  'text'
done
```

Figure 43. Writing to the Transcript

Read streams

ReadStream on: aCollection reads the elements of an Array, String or other sequenceable collection in order.

Selector	Meaning
next next: peek atEnd	Read one element, several elements, look at the next element, test for the end. next at the end is an error.
upTo: upToEnd	Read up to (and past) an element, or to the end
skip: skipSeparators position position: reset setToEnd	Move
do: collect: select:	Enumerate the rest of the stream
contents size isEmpty	The whole collection

Figure 44. Methods of ReadStream

Write streams

WriteStream on: String new collects characters; WriteStream on: Array new collects any objects. contents answers what has been written. A write stream accepts nextPut:, nextPutAll:, print:, display:, <<, nl, tab and space. It is how printOn: methods build their text.

```
| ws rs |
ws := WriteStream on: String new.
#('alpha' 'beta' 'gamma') do: [:w | ws nextPutAll: w] separatedBy: [ws nextPutAll: ', '].
ws contents displayNl.
ws := WriteStream on: Array new.
1 to: 5 do: [:i | ws nextPut: i * i].
ws contents printNl.
rs := ReadStream on: 'one two three'.
[rs atEnd] whileFalse: [(rs upTo: $ ) displayNl].

alpha, beta, gamma
(1 4 9 16 25 )
one
two
three
```

Figure 45. Streams

Standard input

Smalltalk readLine answers the next line of standard input as a String, or nil at the end of the input; Smalltalk stdinAtEnd tests for the end. The example wordfreq.st reads text this way.

Files

FileStream read: path, write: path and append: path open a file. A read stream holds the whole file; nextLine, next, atEnd, upTo: and upToEnd read it. A write stream collects what is written and sends it to the file on flush or close, so always close it. FileStream exists: path tests for a file and FileStream contentsOf: path answers its text, or nil. Opening a missing file for reading is an error of class NotFound.

```
| f |
f := FileStream write: '/tmp/demo.txt'.
f nextPutAll: 'line one'; nl; nextPutAll: 'line two'; nl.
f close.
f := FileStream read: '/tmp/demo.txt'.
[f atEnd] whileFalse: [f nextLine displayNl].
(FileStream exists: '/tmp/nope') printNl.

line one
line two
false
```

Figure 46. Writing and reading a file

Date and Time

Date today and Time now answer the current date and time. A Date answers year, month and day and prints as 2026-10-10; a Time answers hours, minutes and seconds and prints as 14:05:09. Date year:month:day: and Time hours:minutes:seconds: make particular values, and both compare with < and =. Time millisecondClock answers the local time in milliseconds since 1970-01-01, and Time millisecondsToRun: answers the milliseconds a block takes.

Dates support arithmetic. addDays: answers a new Date, subtractDate: answers the number of days between two dates, and dayOfWeek, dayName, monthName, isLeapYear and daysInMonth describe a date. A Time offers addSeconds: (wrapping at midnight) and subtractTime:, which answers seconds.

```
| d |
d := Date year: 2026 month: 10 day: 10.
d dayName printNl.
(d addDays: 30) printNl.
((Date year: 2028 month: 3 day: 1) subtractDate: (Date year: 2028 month: 2 day: 1)) printNl.
((Time hours: 23 minutes: 59 seconds: 30) addSeconds: 45) printNl.

#Saturday
2026-11-09
29
00:00:15
```

Figure 47. Date and Time arithmetic

addMonths: moves a date by calendar months and shortens the day when the target month is too short. longPrintString gives text such as Saturday, 10 October 2026, and dayOfYear and isWeekend answer as their names say. A DateTime joins a Date and a Time: DateTime now, or DateTime year:month:day:hour:minute:second:. It prints as 2026-12-31T23:59:00, compares with < and =, and supports addSeconds:, addDays: and subtractDateTime:, which answers seconds.

```
| a |
a := DateTime year: 2026 month: 12 day: 31 hour: 23 minute: 59 second: 0.
(a addSeconds: 120) printNl.
((a addSeconds: 3600) subtractDateTime: a) printNl.
(Date year: 2026 month: 10 day: 10) longPrintString displayNl.
((Date year: 2026 month: 1 day: 31) addMonths: 1) printNl.

2027-01-01T00:01:00
3600
Saturday, 10 October 2026
2026-02-28
```

Figure 48. DateTime

More stream methods

A read stream understands skipTo:, detect:, notEmpty, back, lines and linesDo:. A write stream understands crtab, lf, newLine, flush, close and print:paddedWith:to:. Blocks understand cull:, timeToRun and ifError:, and every object understands in:, displayOn:, instVarNamed: and deny:.

Stopwatch

Stopwatch new answers a timer with start, stop, reset, isRunning and elapsed (milliseconds). Stopwatch time: aBlock runs the block and answers the elapsed milliseconds.

Parsing and formatting dates

asDate, asTime and asDateTime read the forms 2026-10-10, 14:05:09 and 2026-10-10T14:05:09; the date separator may also be a slash and the seconds may be left out of a time. Date formatted: and Time formatted: fill in a pattern: yyyy, yy, mm, dd, mon, month and day for a date, and hh, mm and ss for a time.

```
'2026-10-10' asDate printNl.
'14:05' asTime printNl.
('2026-10-10' asDate formatted: 'day, dd mon yyyy') displayNl.
('14:05:09' asTime formatted: 'hh.mm') displayNl.

2026-10-10
14:05:00
Saturday, 10 Oct 2026
14.05
```

Figure 49. Parsing and formatting

ReadWriteStream

A ReadWriteStream is written with nextPut: and nextPutAll: and read with next, next:, peek, upTo:, upToEnd and nextLine. reset moves the read position back to the start and setToEnd moves it to the end; contents answers everything written.

```
| rw |
rw := ReadWriteStream on: String new.
rw nextPutAll: 'hello world'.
rw reset.
(rw next: 5) printNl.
rw upToEnd printNl.

'hello'
' world'
```

Figure 50. Writing, then reading

Managing files and directories

FileStream delete: path removes a file, FileStream rename: old to: new renames it and FileStream copy: old to: new copies it; each answers true on success. FileStream size: path answers the size in bytes, or nil if the file does not exist, and FileStream isFile: path tests for a file. FileStream write: path contents: text and FileStream append: path contents: text write or append a whole String in one step, and FileStream lines: path answers the lines of a file.

The class Directory has only class-side messages. Directory exists: path tests for a directory, create: makes one and remove: deletes an empty one. entries: answers the sorted names inside a directory, fileNames: only the files and directoryNames: only the sub-directories; a directory that cannot be read is a NotFound error. Directory current answers the working directory and current: changes it.

```
(Directory create: '/tmp/demo_dir') printNl.
(FileStream write: '/tmp/demo_dir/a.txt' contents: 'hello') printNl.
(Directory entries: '/tmp/demo_dir') printNl.
(FileStream size: '/tmp/demo_dir/a.txt') printNl.
(FileStream delete: '/tmp/demo_dir/a.txt') printNl.
(Directory remove: '/tmp/demo_dir') printNl.

true
5
('a.txt' )
5
true
true
```

Figure 51. Files and directories

Running programs

Smalltalk systemOutput: command runs a shell command and answers what it wrote to standard output as a String, without the final newline. Smalltalk system: command runs the command, shows its output on the Transcript and answers its exit status. lastExitCode answers the status of the latest command and systemError the text it wrote to standard error. Smalltalk getenv: and Smalltalk arguments read the environment and the command-line arguments.

```
(Smalltalk systemOutput: 'echo hi; echo there') printNl.
Smalltalk lastExitCode printNl.
(Smalltalk system: 'exit 3') printNl.

'hi
there'
0
3
```

Figure 52. Running a command

CSV

CSV parse: text answers an Array of rows, each an Array of Strings. Fields may be quoted with double quotes, a doubled quote stands for one quote, and a quoted field may contain commas and line ends. CSV parseFile: reads a file and CSV parseWithHeader: answers an Array of Dictionaries keyed by the first row. CSV format: rows answers CSV text, quoting fields where needed, and CSV write: rows to: path writes it to a file. asCsvRows and asCsv are the same operations as messages to a String and to a collection.

```
(CSV parse: 'a,b
1,"x, y"') printNl.
(CSV format: #(#('a' 'b c') #('x,y' 'say "hi"'))) displayString displayNl.

(('a' 'b' ) ('1' 'x, y' ) )
a,b c
"x,y","say ""hi"""
```

Figure 53. Reading and writing CSV

Chapter 9. Networking

Smalltalk/CREXX reaches the network through the rxsocket library of CREXX. The class Socket is a TCP socket, URL takes web addresses apart and HttpClient fetches pages. All of it is written for one process: a call waits until the network answers or the time-out (ten seconds for a Socket, fifteen for HttpClient) has passed.

Sockets

Socket connectTo: host port: n opens a connection and Socket secureConnectTo: host port: n opens a connection with TLS. Socket listenOn: n makes a server socket on the local host (127.0.0.1); accept waits for a client and answers a new Socket for it. send: writes a String and sendLine: writes it with a line end (carriage return and line feed). receive: n answers up to n characters, receiveLine answers the next line without its line end (nil at the end of the stream), receiveExactly: n waits for n characters and receiveAll reads until the other side closes. All of these first use characters already received. close ends the connection. timeout: sets the time-out in milliseconds; peer, localAddress, status and lastError describe the connection. A failure signals an Error whose text names the operation and the reason.

A connection to the same machine completes as soon as the server is listening, so one program can play both roles: connect first, then accept.

```
| srv cli acc |
srv := Socket listenOn: 19878.
cli := Socket connectTo: '127.0.0.1' port: 19878.
acc := srv accept.
cli sendLine: 'ping'.
acc receiveLine displayNl.
acc sendLine: 'pong'.
cli receiveLine displayNl.
cli close. acc close. srv close.

ping
pong
```

Figure 54. A server and a client in one program

URLs

URL fromString: s answers an object that knows scheme, host, port, path and query. The port is that of the scheme (80 for http, 443 for https) unless the address gives one; the path is '/' if there is none. isSecure tests for https and requestTarget answers the path with the query.

```
| u |
u := URL fromString: 'https://example.com/a/b?x=1'.
u host printNl.
u port printNl.
u path printNl.
u query printNl.
(URL fromString: 'http://localhost:8080') port printNl.

'example.com'
443
'/a/b'
'x=1'
8080
```

Figure 55. Taking an address apart

HTTP

HttpClient get: url and HttpClient post: url body: text answer an HttpResponse; HttpClient getString: url answers only the body. The client sends a HTTP/1.1 request with Connection: close, joins the chunks of a

chunked answer and understands https. An HttpResponse answers status (an Integer), reason, body, headers (a Dictionary with lower case keys), header: name (nil if missing) and isSuccess for the codes 200 to 299. HttpClient parseResponse: text builds a response from the raw text of an answer, which is useful for tests.

```
| r |
r := HttpClient parseResponse: 'HTTP/1.1 200 OK', HttpClient crlf, 'Content-Type:
text/plain', HttpClient crlf, HttpClient crlf, 'hi'.
r status printNl.
r body printNl.
(r header: 'content-type') printNl.
r isSuccess printNl.

200
'hi'
'text/plain'
true
```

Figure 56. Reading a response

The sqlite plugin of CREXX is not reachable from Smalltalk/CREXX; it is not part of every CREXX installation.

Chapter 10. Testing with SUnit

The classes TestCase, TestSuite and TestResult are a small version of SUnit, the unit test framework of Smalltalk. A test is a method whose name begins with test in a subclass of TestCase. For each test the framework makes a new instance, sends setUp, runs the method and sends tearDown, even when the test failed.

Writing tests

Inside a test the following messages check results. assert: aBoolean fails if the argument is false and deny: if it is true; assert: a equals: b fails if a = b is false and says what was expected. should: aBlock raise: ExceptionClass fails if the block does not signal that exception, shouldnt:raise: fails if it does. fail and fail: text fail at once. A failure is an error of class TestFailure; any other error in a test is counted as an error, not a failure.

Running tests

MyTest run answers a TestResult with the counts and lists passed, failures and errors. MyTest runAndReport also writes a report on the Transcript with one line for each test that did not pass. MyTest suite answers a TestSuite of the tests, including those inherited from superclasses (below TestCase); MyTest run: #testName runs one test. A TestSuite is built with TestSuite named: and addTest:, and its run answers a TestResult.

```
TestCase subclass: StackTest [
    | s |
    setUp [ s := OrderedCollection new. s add: 1; add: 2 ]
    testSize [ self assert: s size equals: 2 ]
    testLast [ self assert: s last equals: 3 ]
    testRaise [ self should: [1/0] raise: ZeroDivide ]
    testError [ ^nil foo ]
]
StackTest runAndReport.

StackTest: 4 run, 2 passed, 1 failed, 1 errors
  FAIL StackTest>>#testLast: Expected 3 but got 2
  ERROR StackTest>>#testError: did not understand #foo
```

Figure 57. A test case

Selector	Meaning
assert: deny:	Fail if the argument is false; fail if it is true
assert:equals:	Fail if the two objects are not equal
should:raise: shouldnt:raise:	Check that a block does, or does not, signal an exception
fail fail:	Fail the test
setUp tearDown	Override to make and to release the fixture
run runAndReport suite run:	Class side: run all tests, run and report, answer the suite, run one test

Figure 58. Messages of TestCase

Chapter 11. Processes

Smalltalk/CREXX has Process, Semaphore, SharedQueue and Delay and a global Processor, but there is only one real thread of control. Processes are cooperative: a process runs until it ends or has to wait. Waiting is done by running the other ready processes inside the wait, which suits programs where a process waits for another to produce something, and does not suit programs that switch back and forth between two processes with yield.

Starting processes

aBlock fork makes a process with the priority of the running process and puts it in the ready queue; it starts when the running process waits or yields, or when the current top-level statement (or Eval) ends. forkAt: priority gives a priority; a process with a higher priority than the running one starts at once. newProcess makes a suspended process which resume starts. Processor activeProcess answers the running process. The priorities are Processor idlePriority (1) to highestPriority (9); user programs run at userSchedulingPriority (4); userBackgroundPriority is 3 and userInterruptPriority is 5. A Process answers priority, name, name:, state, isTerminated, isSuspended, resume, suspend and terminate. terminate runs the ensure: blocks. An error that is not handled in a process is reported on the Transcript and ends that process only.

```
| log |
log := OrderedCollection new.
Eval [
    [log add: 'low'] forkAt: Processor userBackgroundPriority.
    [log add: 'same'] fork.
    [log add: 'high'] forkAt: Processor userInterruptPriority.
    log add: 'main'.
    Processor yield.
    log add: 'after yield'
]
log printNl.

OrderedCollection ('high' 'main' 'same' 'after yield' 'low' )
```

Figure 59. Fork and priorities

Semaphores, queues and delays

A Semaphore counts signals. signal adds one; wait takes one and, if there is none, lets the other processes run until there is one. Semaphore forMutualExclusion makes a semaphore with one signal, and critical: aBlock runs the block with the semaphore held. A SharedQueue passes objects between processes: nextPut: adds one and next waits for one. Delay forMilliseconds: n and Delay forSeconds: n answer a delay whose wait lets other processes run for that time and then sleeps. A wait that no other process can end is a deadlock; it is reported as an Error and the statement is abandoned.

```
| q results |
q := SharedQueue new.
results := OrderedCollection new.
[1 to: 5 do: [:i | q nextPut: i * i]. q nextPut: nil] fork.
[| v | [(v := q next) notNil] whileTrue: [results add: v]] fork.
Processor yield.
results printNl.

OrderedCollection (1 4 9 16 25 )
```

Figure 60. Producer and consumer

Chapter 12. Exceptions

Handling

aBlock on: ExceptionClass do: [:e | ...] evaluates the block. If an exception of that class (or a subclass) is signaled while it runs, the handler block is evaluated with the exception as its argument. The value of the handler block becomes the value of the on:do: expression. A handler for several classes uses a set: on: ZeroDivide, MessageNotUnderstood do: [...].

```
| r |
r := [10 / 0] on: ZeroDivide do: [:e | e messageText displayNl. -1].
r printNl.
r := [nil foo] on: MessageNotUnderstood do: [:e | e return: 3].
r printNl.
r := [#(1 2) at: 5] on: Error do: [:e | e messageText].
r displayNl.

The program attempted to divide a number by zero
-1
3
Index value 5 out of range: only 2 elements
```

Figure 61. Catching an error

Messages understood by an exception

Selector	Meaning
messageText description	The text given to signal:, or the description of the class
return return:	Leave on:do: with nil or a value
retry retryUsing:	Run the protected block again; or leave with the value of another block
resume resume:	Continue after the signal with a value. Only resumable exceptions (not Error) may be resumed.
pass outer signal	Pass to the enclosing handler (pass does not come back; outer does if the outer handler resumes); signal again
isResumable receiver class	Query

Figure 62. Methods of Exception

Signaling

Error signal: 'text', Error new signal: 'text' and self error: 'text' signal an Error. ExceptionClass signal and signal: are class-side shortcuts. The exception classes are the following.

```
Exception
  Error
    ArithmeticError
      ZeroDivide
    MessageNotUnderstood
    InvalidValue
      InvalidArgument
        IndexOutOfRange
        NotFound
      WrongClass
    EmptyCollection
  Warning
```

Figure 63. The exception classes

An Error is not resumable. A Warning and a plain Exception are resumable: if no handler is active, a Warning prints its trace and continues, returning nil from signal.

Cleaning up

aBlock ensure: cleanup evaluates cleanup after aBlock, however aBlock ends: normally, by an exception or by a non-local return. aBlock ifCurtailed: cleanup evaluates cleanup only if aBlock does not finish normally.

```
| n r |
r := [(Warning signal: 'careful') printNl. 'done'] on: Warning do: [:e | e resume: 42].
r printNl.
n := 0.
r := [n := n + 1. n < 3 ifTrue: [Error signal: 'again']. n] on: Error do: [:e | e retry].
r printNl.
r := [[Error signal: 'inner'] ensure: ['cleanup' displayNl]] on: Error do: [:e | 'handler'
displayNl. 9].
r printNl.

42
'done'
3
handler
cleanup
9
```

Figure 64. Resume, retry and ensure

Notice that the handler runs before the stack is unwound, so the handler text appears before the cleanup text.

Your own exceptions

```
Error subclass: InsufficientFunds [
    | needed |
    needed [ ^needed ]
    needed: n [ needed := n ]
    messageText [ ^'Insufficient funds: need ', needed printString, ' more' ]
]
[InsufficientFunds new needed: 430; signal]
    on: InsufficientFunds do: [:e | e messageText displayNl].
InsufficientFunds new needed: 5; signal.

Insufficient funds: need 430 more
Object: nil error: Insufficient funds: need 5 more
UndefinedObject>>executeStatements
```

Figure 65. A user-defined exception

Chapter 13. Reflection and the System

Asking about objects

Selector	Meaning
class	The class of the receiver
isKindOf: isMemberOf: respondsTo:	Class membership; whether a message is understood
isNil notNil isString isSymbol isNumber isInteger isCharacter isArray isClass	Type tests
== = ~= ~~ hash identityHash	Identity and equality
copy shallowCopy deepCopy postCopy	Copying; copy sends postCopy to the shallow copy
perform: perform:with: perform:withArguments:	Send a message chosen at run time
instVarAt: instVarAt:put:	Access an instance variable by position
yourself ->	The receiver; make an Association

Figure 66. Reflection messages

Asking about classes

Selector	Meaning
name printString	The name of the class
superclass subclasses allSubclasses allSuperclasses	Position in the hierarchy
selectors includesSelector: canUnderstand:	The methods of a class
instanceVariableNames allInstVarNames instSize	Instance variables
inheritsFrom: includesBehavior:	Inheritance tests
subclass: subclass:instanceVariableNames:classVariableNames:package:	Create a subclass
addInstVarName: addClassVarName: instanceVariableNames:	Add variables

Figure 67. Class messages

```
Object subclass: Probe [ |a| a [ ^a ] ]
Probe selectors printNl.
Probe instanceVariableNames printNl.
(Probe new respondsTo: #a) printNl.
(Probe new respondsTo: #zork) printNl.
(3 perform: #+ with: 4) printNl.
(Probe new isKindOf: Object) printNl.
Probe superclass printNl.

(#a )
(#a )
true
false
7
true
Object
```

Figure 68. Reflection

The Smalltalk dictionary

Smalltalk is the dictionary of global variables. Smalltalk at: #Name put: value creates or sets a global, Smalltalk at: #Name reads it, and Smalltalk includesKey: #Name tests for it. A global created this way can

then be used by its plain name, Name, in any program. Smalltalk version answers the version string. Smalltalk halt and Smalltalk quit: n end the program.

Smalltalk getenv: name answers the value of an environment variable, or an empty String when it is not set. Smalltalk exit and Smalltalk exit: n end the program with a status. Random seed: n restarts the random sequence from a given seed, so that a program can repeat its numbers exactly.

Contexts and compiling at run time

thisContext answers a Context for the running method. A Context answers receiver, selector, methodClass, sender (the Context of the caller, nil at the top), depth, backtrace (which writes the stack on the Transcript) and backtraceString. It is a snapshot taken when thisContext is evaluated; it cannot be used to change the stack.

Smalltalk evaluate: aString evaluates the statements in the String and answers the value of the last one. aClass compile: aString adds a method to the class: the String is a method in the syntax of a class body, such as 'double: x [^x * 2]', or the old form with the pattern on the first line and the body after it. A class-side method is compiled by sending compile: to the metaclass (A class compile: ...). Smalltalk fileIn: aString reads class definitions and statements from a String, as if they were a file.

```
Object subclass: A [ ]
A compile: 'double: x [ ^x * 2 ]'.
A class compile: 'make [ ^self new ]'.
(A make double: 21) printNl.
(Smalltalk evaluate: '3 + 4') printNl.
Object subclass: B [ foo [ ^self bar ] bar [ ^thisContext ] ]
B new foo printNl.
B new foo sender printNl.

42
7
B>>bar
B>>foo
```

Figure 69. Compiling and evaluating

Chapter 14. Errors and Traces

Unhandled errors

When an exception is signaled and no handler is active, the interpreter prints a line that names the object and the error, and then a trace of the active methods, innermost first, ending with UndefinedObject>>executeStatements. The statement is abandoned and the program continues with the next one. This is the form used by GNU Smalltalk.

```
Object subclass: T [
    a [ ^self b ]
    b [ ^1 / 0 ]
]
T new a.
'next' displayNl.

Object: 1 error: The program attempted to divide a number by zero
SmallInteger(Number)>>zeroDivide
SmallInteger(Number)>>/
T>>b
T>>a
UndefinedObject>>executeStatements
next
```

Figure 70. A trace

A trace line reads Receiver(ImplementingClass)>>selector. When the receiver's class is also the class that defines the method, the part in parentheses is left out. Blocks that were compiled in line, and the lines of the exception machinery itself, do not appear. ensure: blocks run after the trace is printed.

Common messages

Message	Cause
did not understand #selector	A message was sent that the receiver does not understand. For an argument of an arithmetic message the missing selector is #generality.
The program attempted to divide a number by zero	/ // \\ rem: or quo: with a zero divisor
Index value n out of range: only m elements	at: or at:put: with an index outside a collection
key not found	Dictionary at: or removeKey: of a missing key
access beyond end of object	removeFirst, removeLast or first on an empty OrderedCollection
tried to access an empty collection	first, last, anyOne or fold: on any other empty collection
This method is a subclass responsibility	subclassResponsibility was sent
wrong argument count	A block was evaluated with the wrong number of arguments
call stack too deep	More than 20000 nested method calls
BlockContext>>cannotReturn:	A block with ^ was evaluated after its method had returned

Figure 71. Error messages

Parse errors

A syntax error is reported as Parse error (file line n): text. The interpreter then skips to the end of the statement and goes on.

Chapter 15. How Smalltalk/CREXX Works

Objects

A reference to an object (an oop) is an integer. An odd integer 2n+1 stands for the SmallInteger n and has no storage. An even integer 2i is the index i of an entry in the object table, which is a set of parallel arrays: class, kind, number of slots, start of the slots in a common slot pool, a numeric field, a string field and a float field. nil, true and false are entries 1, 2 and 3.

The kind of an entry says how the native fields are used: plain object, String, Symbol, Float, Array, Character, block, large integer, Dictionary, class or metaclass. A String holds its text in the string field, so that string operations are done by CREXX built-in functions, and a large integer holds its decimal digits in the same field. A Dictionary holds a hash map from key text to a linked entry, which gives constant-time access and keeps insertion order.

Classes and methods

Each class has a number. Its record holds the superclass, the metaclass, the names of the instance variables including those it inherits, and the kind of instances it creates. Methods are kept in a table indexed by class number and selector number. Each selector name gets a number the first time it is seen. A cache of lookup results is flushed whenever a method is installed.

The resolver

A method body is parsed into a raw tree. The resolver walks it once and replaces names by their meaning: a local variable becomes (levels up, slot), an instance variable becomes its position in the object, a class variable or global becomes a cell number. The control-flow messages with literal block arguments become single nodes, and their variables live in the frame of the enclosing method or block, so that no block object is created for them.

The evaluator

Frames are kept in one array. A frame holds the outer frame, the home (method) frame, self, the method, and then the arguments and temporaries. Frames of methods and blocks that create no closures are returned to a free list as soon as they finish; the others are left in place because a block may still refer to them. The arguments of a send are placed on a separate stack.

Returns and exceptions are done by unwinding. The evaluator checks a global flag after every call; a ^ in a block sets it to return-to-this-frame and the frames above are abandoned in turn. Exceptions use the same mechanism: signal finds a handler by searching a stack of active handlers and runs its block on top of the stack that signaled, so that the handler can resume; return, retry and pass then unwind to the on:do: that installed the handler.

Primitives

A method may start with <primitive: name>. The named CREXX routine is run first; if it returns a value that is the result of the method, and if it fails the Smalltalk statements that follow the pragma are run. This is how the library provides, for example, SmallInteger + with fallbacks for fractions and errors. Arithmetic and comparison of two SmallIntegers is done directly by the send routine without looking up a method.

Adding a primitive

1. Add the name to src/prims.txt. Its position gives it a number, and the constant PR_NAME.
2. Add a branch for PR_NAME to prim_a, prim_b or prim_c in src/f_prim.crexx. The receiver is rcv, the arguments are ast[base+1] to ast[base+na]. Return 0 to fail.
3. Use it in the library: method [<primitive: name> ^fallback].
4. Run python3 build.py and tests/run_tests.sh.

The build

build.py joins the files in src in a fixed order, replaces the symbolic constants (token types TK_, node types ND_, object kinds KD_, unwind kinds UW_, selectors S_, primitives PR_) by numbers, fills in the list of global variables of the namespace, and writes smalltalk.crexx.

Chapter 16. Restrictions and Known Problems

- **new does not call initialize.** Override the class-side new, as the examples do.
- **Dictionary keys** are compared by built-in equality for numbers, strings, symbols, characters and arrays of those, and by identity for other objects. A user-defined = and hash are not consulted by Dictionary, Set or Bag.
- **No garbage collector.** Objects are never freed. A program that creates many millions of objects will use a great deal of memory.
- **Speed.** The interpreter is a program running on a virtual machine. About one million simple sends per second can be expected, and a million-iteration loop with a simple body takes about a second; the start-up, which reads the library, takes a second or two. The directory bench of the repository holds the benchmarks (bench/run_bench.sh) used to measure speed. Speed depends mostly on the number of global variables of the interpreter, because the CREXX virtual machine relinks all of them at every procedure call; build.py therefore packs the interpreter's arrays and variables into a few container arrays (src/pack.txt). Large-integer arithmetic works on decimal strings and is slow for numbers of thousands of digits.
- **Floats** have the precision of the CREXX floating-point type, about 15 digits, and print with that many. The printed form of results such as 0.1 + 0.2 is 0.3.
- **SmallInteger** covers 18 decimal digits (SmallInteger maxVal is 999999999999999999); larger values are large integers.
- **No processes** (fork, Semaphore, Process) and no thisContext. file streams read and write whole files.
- **Class definitions** must name an existing superclass. The bang chunk format of old Smalltalk-80 files is not supported.
- **Namespaces.** A dotted name such as SystemExceptions.NotFound is read as the last part only.
- **Strings** are byte strings: characters have codes 0 to 255.
- **String comparison** with < is by character code, not case-insensitive.

Appendix A. Command Summary

```
./smalltalk [-i] [file.st ...] [-- arg ...]
crexx smalltalk.crexx -nocompile -args -L lib [-i] [file.st ...] [-- arg ...]

-L dir   directory of the library files (the launcher supplies it)
-i       run the files, then an interactive prompt
--       the words that follow are program arguments
-h       show usage
```

Figure 72. Command line

With no file name the program is read from standard input. The environment variable CREXX names the crexx driver used by the launcher.

Appendix B. Contents of the Repository

Path	Contents
smalltalk	Launcher script.
smalltalk.crexx	The interpreter, generated by build.py (checked in so that the repository runs at once).
build.py	Assembles smalltalk.crexx from src.
src/	The interpreter sources, consts.txt, selectors.txt and prims.txt.
lib/	The class library in Smalltalk: kernel.st, numbers.st, collections.st, streams.st, exceptions.st.
examples/	Sample programs.
tests/	Regression tests and run_tests.sh.
docs/	This manual as a Word document and as a PDF.
README.md	Short description of the project.

Figure 73. Repository

Appendix C. Class Hierarchy

```
Object
  Behavior
    ClassDescription
      Class
      Metaclass
  Boolean
    True
    False
  UndefinedObject
  Magnitude
    Character
    Number
      Integer
        SmallInteger
        LargeInteger
          LargePositiveInteger
          LargeNegativeInteger
      Float
        FloatD
      Fraction
      Complex
    LookupKey
      Association
    Date, Time, DateTime
  Collection
    SequenceableCollection
      ArrayedCollection
        Array
        CharacterArray
          String
            Symbol
        Interval
      OrderedCollection
        SortedCollection
    HashedCollection
      Dictionary
        IdentityDictionary
      Set
        IdentitySet
      Bag
    LinkedList
  Stack, Queue, Matrix, LinkNode, JsonParser, Stopwatch
  Stream
    PositionableStream
      ReadStream
      WriteStream
        ReadWriteStream
          FileStream
    TextCollector
  BlockClosure, Message, Point, Random, SystemDictionary
  Signal
    Exception
      Error, Warning
```

Figure 74. The kernel classes

Exception classes are shown in Chapter 12.

Appendix D. Sample Programs

The directory examples holds the programs in the figure below.

File	Shows
hello.st	Transcript, printNl
fib.st	Class-side methods, recursion, large integers
shapes.st	Inheritance, polymorphism, printOn:, sorted collections, fractions
bank.st	User-defined exceptions, on:do:, ensure:, error:
primes.st	Array, loops, select: on an Interval
hanoi.st	Recursive blocks
streams.st	WriteStream, ReadStream, strings
wordfreq.st	Dictionary, standard input, sorting with a sort block

Figure 75. Sample programs

The regression tests tests/01_classes_basic.st to tests/07_objects.st exercise classes, collections, strings, numbers, exceptions and the object model, and show the exact output, traces included.